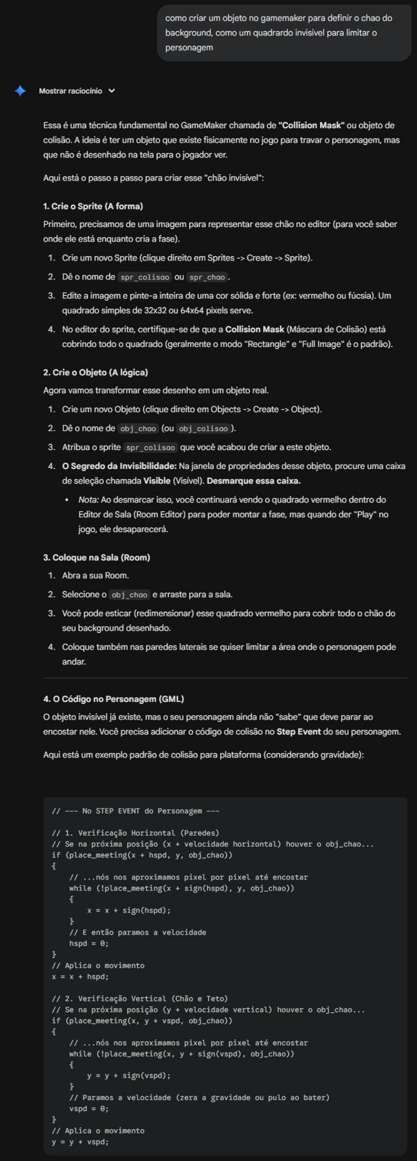


como criar um objeto no gamemaker para definir o chao do background, como um quadrardo invisivel para limitar o personagem

Mostrar raciocínio

Essa é uma técnica fundamental no GameMaker chamada de **"Collision Mask"** ou objeto de colisão. A ideia é ter um objeto que existe fisicamente no jogo para travar o personagem, mas que não é desenhado na tela para o jogador ver.

Aqui está o passo a passo para criar esse "chão invisível":

1. Crie o Sprite (A forma)

Primeiro, precisamos de uma imagem para representar esse chão no editor (para você saber onde ele está enquanto cria a fase).

1. Crie um novo Sprite (clique direito em Sprites -> Create -> Sprite).
2. Dê o nome de `spr_colisao` ou `spr_chao`.
3. Edite a imagem e pinte-a inteira de uma cor sólida e forte (ex: vermelho ou fúcsia). Um quadrado simples de 32x32 ou 64x64 pixels serve.
4. No editor do sprite, certifique-se de que a **Collision Mask** (Máscara de Colisão) está cobrindo todo o quadrado (geralmente o modo "Rectangle" e "Full Image" é o padrão).

2. Crie o Objeto (A lógica)

Agora vamos transformar esse desenho em um objeto real.

1. Crie um novo Objeto (clique direito em Objects -> Create -> Object).
2. Dê o nome de `obj_chao` (ou `obj_colisao`).
3. Atribua o sprite `spr_colisao` que você acabou de criar a este objeto.
4. **O Segredo da Invisibilidade:** Na janela de propriedades desse objeto, procure uma caixa de seleção chamada **Visible** (Visível). **Desmarque essa caixa.**
    - *Nota:* Ao desmarcar isso, você continuará vendo o quadrado vermelho dentro do Editor de Sala (Room Editor) para poder montar a fase, mas quando der "Play" no jogo, ele desaparecerá.

3. Coloque na Sala (Room)

1. Abra a sua Room.
2. Selecione o `obj_chao` e arraste para a sala.
3. Você pode esticar (redimensionar) esse quadrado vermelho para cobrir todo o chão do seu background desenhado.
4. Coloque também nas paredes laterais se quiser limitar a área onde o personagem pode andar.

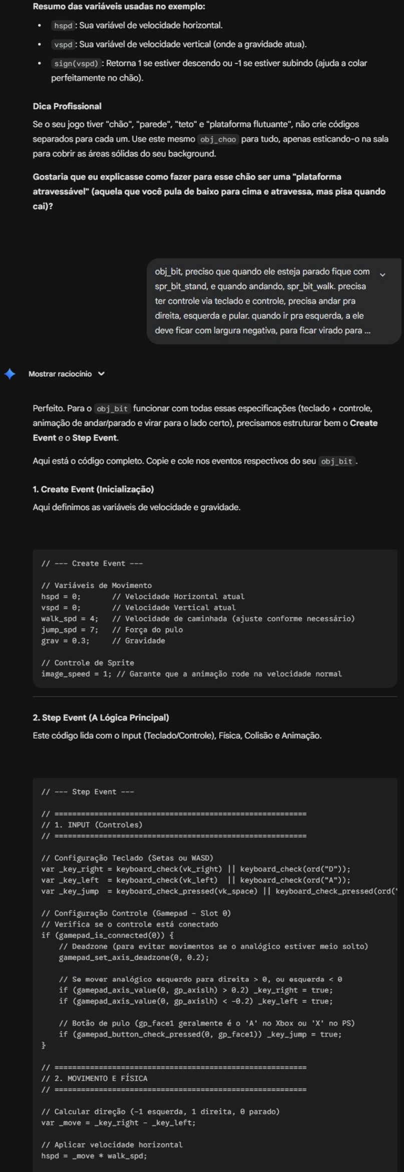
---

4. O Código no Personagem (GML)

O objeto invisível já existe, mas o seu personagem ainda não "sabe" que deve parar ao encostar nele. Você precisa adicionar o código de colisão no **Step Event** do seu personagem.

Aqui está um exemplo padrão de colisão para plataforma (considerando gravidade):

```
// --- No STEP EVENT do Personagem ---

// 1. Verificação Horizontal (Paredes)
// Se na próxima posição (x + velocidade horizontal) houver o obj_chao...
if (place_meeting(x + hspd, y, obj_chao))
{
    // ...nós nos aproximamos pixel por pixel até encostar
    while (!place_meeting(x + sign(hspd), y, obj_chao))
    {
        x = x + sign(hspd);
    }
    // E então paramos a velocidade
    hspd = 0;
}
// Aplica o movimento
x = x + hspd;

// 2. Verificação Vertical (Chão e Teto)
// Se na próxima posição (y + velocidade vertical) houver o obj_chao...
if (place_meeting(x, y + vspd, obj_chao))
{
    // ...nós nos aproximamos pixel por pixel até encostar
    while (!place_meeting(x, y + sign(vspd), obj_chao))
    {
        y = y + sign(vspd);
    }
    // Paramos a velocidade (zera a gravidade ou pulo ao bater)
    vspd = 0;
}
// Aplica o movimento
y = y + vspd;
```



Resumo das variáveis usadas no exemplo:

- `hspd`: Sua variável de velocidade horizontal.
- `vspd`: Sua variável de velocidade vertical (onde a gravidade atua).
- `sign(vspd)`: Retorna 1 se estiver descendo ou -1 se estiver subindo (ajuda a colar perfeitamente no chão).

Dica Profissional

Se o seu jogo tiver "chão", "parede", "teto" e "plataforma flutuante", não crie códigos separados para cada um. Use este mesmo `obj_chao` para tudo, apenas esticando-o na sala para cobrir as áreas sólidas do seu background.

Gostaria que eu explicasse como fazer para esse chão ser uma "plataforma atravessável" (aquela que você pula de baixo para cima e atravessa, mas pisa quando cai)?

obj_bit, preciso que quando ele esteja parado fique com spr_bit_stand, e quando andando, spr_bit_walk. precisa ter controle via teclado e controle, precisa andar pra direita, esquerda e pular. quando ir pra esquerda, a ele deve ficar com largura negativa, para ficar virado para ...

Perfeito. Para o `obj_bit` funcionar com todas essas especificações (teclado + controle, animação de andar/parado e virar para o lado certo), precisamos estruturar bem o **Create Event** e o **Step Event**.

Aqui está o código completo. Copie e cole nos eventos respectivos do seu `obj_bit`.

1. Create Event (Inicialização)

Aqui definimos as variáveis de velocidade e gravidade.

```
// --- Create Event ---

// Variáveis de Movimento
hspd = 0;       // Velocidade Horizontal atual
vspd = 0;       // Velocidade Vertical atual
walk_spd = 4;   // Velocidade de caminhada (ajuste conforme necessário)
jump_spd = 7;   // Força do pulo
grav = 0.3;     // Gravidade

// Controle de Sprite
image_speed = 1; // Garante que a animação rode na velocidade normal
```

2. Step Event (A Lógica Principal)

Este código lida com o Input (Teclado/Controle), Física, Colisão e Animação.

```
// --- Step Event ---

// =========================================================
// 1. INPUT (Controles)
// =========================================================

// Configuração Teclado (Setas ou WASD)
var _key_right = keyboard_check(vk_right) || keyboard_check(ord("D"));
var _key_left  = keyboard_check(vk_left)  || keyboard_check(ord("A"));
var _key_jump  = keyboard_check_pressed(vk_space) || keyboard_check_pressed(ord("

// Configuração Controle (Gamepad - Slot 0)
// Verifica se o controle está conectado
if (gamepad_is_connected(0)) {
    // Deadzone (para evitar movimentos se o analógico estiver meio solto)
    gamepad_set_axis_deadzone(0, 0.2);

    // Se mover analógico esquerdo para direita > 0, ou esquerda < 0
    if (gamepad_axis_value(0, gp_axislh) > 0.2) _key_right = true;
    if (gamepad_axis_value(0, gp_axislh) < -0.2) _key_left = true;

    // Botão de pulo (gp_face1 geralmente é o 'A' no Xbox ou 'X' no PS)
    if (gamepad_button_check_pressed(0, gp_face1)) _key_jump = true;
}

// =========================================================
// 2. MOVIMENTO E FÍSICA
// =========================================================

// Calcular direção (-1 esquerda, 1 direita, 0 parado)
var _move = _key_right - _key_left;

// Aplicar velocidade horizontal
hspd = _move * walk_spd;
```

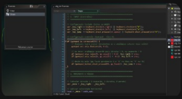
```
// Aplicar Gravidade
vspd = vspd + grav;

// Lógica de Pulo (Só pula se estiver tocando no chão)
if (place_meeting(x, y + 1, obj_chao) && _key_jump) {
    vspd = -jump_spd;
}

// ========================================================
// 3. COLISÃO (O código do passo anterior)
// ========================================================

// Colisão Horizontal
if (place_meeting(x + hspd, y, obj_chao)) {
    while (!place_meeting(x + sign(hspd), y, obj_chao)) {
        x = x + sign(hspd);
    }
    hspd = 0;
}
x = x + hspd;

// Colisão Vertical
if (place_meeting(x, y + vspd, obj_chao)) {
    while (!place_meeting(x, y + sign(vspd), obj_chao)) {
        y = y + sign(vspd);
    }
    vspd = 0;
}
y = y + vspd;

// ========================================================
// 4. ANIMAÇÃO E ESPELHAMENTO
// ========================================================

// Se estiver se movendo (velocidade horizontal diferente de 0)
if (hspd != 0) {
    // Muda para o sprite de andar
    sprite_index = spr_bit_walk;

    // Vira o personagem
    // sign(hspd) retorna 1 (direita) ou -1 (esquerda)
    image_xscale = sign(hspd);
}
else {
    // Se estiver parado
    sprite_index = spr_bit_stand;
    // Nota: Não mudamos o image_xscale aqui para ele continuar olhando
    // para a última direção que andou.
}
```

Explicação dos Pontos Chave

1. **Suporte a Gamepad:** O código verifica `gamepad_is_connected(0)`. O GameMaker reconhece automaticamente controles de Xbox/PlayStation genéricos. O `gp_face1` é o botão de ação inferior (A no Xbox, X no PlayStation).
2. `image_xscale = sign(hspd)`: Essa é a mágica do espelhamento.
    - Se `hspd` for positivo (direita), o `image_xscale` vira `1` (normal).
    - Se `hspd` for negativo (esquerda), o `image_xscale` vira `-1` (invertido/espelhado).
3. **Prioridade de Sprite:** O código verifica `if (hspd != 0)`. Isso garante que o sprite de "andar" só ative se ele estiver realmente saindo do lugar. Se ele bater numa parede e o `hspd` zerar (devido à colisão), ele volta para o sprite `stand` automaticamente.

Você gostaria de adicionar um sprite específico para quando ele estiver no ar (pulando/caindo), para não parecer que ele está caminhando no vento?

Fontes



os comandos parecem nao funcionar

Mostrar raciocínio

O problema mais provável é que **as variáveis não foram definidas antes de serem usadas**.

Pela sua imagem, o código está todo no evento **Etapa (Step)**. O GameMaker lê o código de cima para baixo. Quando ele chega na linha 32 (`hspd = _move * walk_spd;`), ele tenta fazer uma conta com `walk_spd`, mas se você não tiver criado essa variável antes, o jogo trava ou simplesmente ignora o comando se houver um erro silencioso.

Você precisa de um evento **Criar (Create)** para "apresentar" essas variáveis ao jogo antes de começar.

Como corrigir:

1. **Olhe para a janela "Eventos"** (na esquerda da sua imagem).
2. Você já tem o evento **Criar** (a lâmpada). Clique nele.
3. Cole o seguinte código dentro desse evento **Criar**:

```
// --- Evento CRIAR (Create) ---
hspd = 0;       // Velocidade horizontal
vspd = 0;       // Velocidade vertical
walk_spd = 4;   // Velocidade de andar (pode aumentar se estiver lento)
jump_spd = 7;   // Força do pulo
grav = 0.3;     // Gravidade
```

Outra verificação importante:

Se você já tinha colocado esse código no *Criar* e ainda não funciona, o problema pode estar na lógica das chaves `{ }` na parte do gamepad na sua imagem.

Na **linha 12**, você abre uma chave `{` para o `if (gamepad_is_connected(0))`. Na **linha 22**, você fecha essa chave `}`.

O problema: As variáveis `_key_right`, `_key_left` e `_key_jump` são definidas como variáveis locais (`var`) nas linhas 6, 7 e 8. Quando o código sai do bloco do gamepad, o GameMaker pode estar resetando ou perdendo a referência correta se a lógica de mistura entre teclado e gamepad não estiver clara.

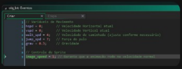
Tente **remover o** `var` das linhas 6, 7 e 8 para torná-las variáveis de instância temporariamente, ou garanta que a lógica do gamepad apenas *atualize* as variáveis já criadas, sem recriá-las.

A forma mais segura e limpa (que funciona para ambos simultaneamente) é esta:

```
// 1. Primeiro define o input do teclado (padrão 0 ou 1)
var _key_right = keyboard_check(vk_right) || keyboard_check(ord("D"));
var _key_left  = keyboard_check(vk_left)  || keyboard_check(ord("A"));
var _key_jump  = keyboard_check_pressed(vk_space) || keyboard_check_pressed(ord('

// 2. DEPOIS verifica o gamepad e SOBRESCREVE se houver input
if (gamepad_is_connected(0)) {
    gamepad_set_axis_deadzone(0, 0.2);
    // Note que aqui usamos "if (...) _key_right = true;" sem usar "var" novament
    if (gamepad_axis_value(0, gp_axislh) > 0.2) _key_right = true;
    if (gamepad_axis_value(0, gp_axislh) < -0.2) _key_left = true;
    if (gamepad_button_check_pressed(0, gp_face1)) _key_jump = true;
}
```

Isso garante que se você apertar o teclado, funciona. Se apertar o controle, funciona também.

Faça o teste preenchendo o evento **Criar** e me avise se o personagem começou a se mover!



```
// Variáveis de Movimento
hspd = 0;       // Velocidade Horizontal atual
vspd = 0;       // Velocidade Vertical atual
walk_spd = 4;   // Velocidade de caminhada (ajuste conforme necessário)
jump_spd = 7;   // Força do pulo
grav = 0.3;     // Gravidade

// Controle de Sprite
image_speed = 1; // Garante que a animação rode na velocidade normal
```

ja ta assim

Mostrar raciocínio

Pelos prints que você mandou, a lógica de entrada (teclado/controle) e a inicialização das variáveis no "Create" parecem **certinhas.** O GameMaker está lendo os controles e calculando a velocidade.

Se o personagem não sai do lugar, geralmente é por um destes três motivos simples. Vamos verificar um por um:

1. O código de *mover* está faltando?

Na imagem do evento **Etapa (Step)**, o código termina na linha 32 (`hspd = ...`). O computador calculou a velocidade, mas **nós ainda não mandamos ele aplicar isso na posição do personagem.**

Verifique se, **abaixo da linha 32**, você tem o restante do código que eu passei antes (a parte que soma `x + hspd`). Se não tiver, copie e cole isso no final do evento Etapa:

```
// ... (seu código vai até a linha 32 aqui)

// Aplicar Gravidade
vspd = vspd + grav;
```

```
// Pulo
if (place_meeting(x, y + 1, obj_chao) && _key_jump) {
    vspd = -jump_spd;
}

// === COLISÃO E MOVIMENTO (Essencial para ele sair do lugar) ===

// Horizontal
if (place_meeting(x + hspd, y, obj_chao)) {
    while (!place_meeting(x + sign(hspd), y, obj_chao)) {
        x = x + sign(hspd);
    }
    hspd = 0;
}
x = x + hspd; // <--- ESTA LINHA que faz ele andar de verdade!

// Vertical
if (place_meeting(x, y + vspd, obj_chao)) {
    while (!place_meeting(x, y + sign(vspd), obj_chao)) {
        y = y + sign(vspd);
    }
    vspd = 0;
}
y = y + vspd; // <--- ESTA LINHA que faz ele cair/pular!

// === ANIMAÇÃO ===
if (hspd != 0) {
    sprite_index = spr_bit_walk;
    image_xscale = sign(hspd);
} else {
    sprite_index = spr_bit_stand;
}
```

---

2. A opção "Uses Physics" está marcada? (Erro comum)

Se você marcou a caixinha **Uses Physics** (Usar Física) nas propriedades do objeto `obj_bit`, esse código manual para de funcionar porque o motor de física tenta assumir o controle.

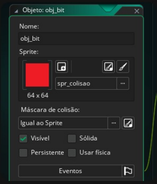
- **Solução:** Abra o `obj_bit` e certifique-se de que a caixa **Uses Physics** está **DESMARCADA**.

3. O "Ponto de Origem" do Sprite

Para a animação de virar para a esquerda (largura negativa) funcionar sem o personagem "teletransportar" para o lado ou ficar preso na parede:

1. Abra seus sprites (`spr_bit_walk` e `spr_bit_stand`).
2. Na configuração de **Origin** (Origem), mude de "Top Left" para **Middle Centre** (Centro Meio) ou **Bottom Centre** (Centro Baixo). Isso faz ele girar em torno do próprio eixo.

Tente adicionar o restante do código (Passo 1) e veja se ele ganha vida!



Objeto: obj_bit

Nome: obj_bit

Sprite: spr_colisao

64 x 64

Máscara de colisão: Igual ao Sprite

☑ Visível ☐ Sólida

☐ Persistente ☐ Usar física

ta desse jeito

Mostrar raciocínio

A boa notícia é que **suas variáveis (Create) e a leitura dos controles (Step) estão perfeitas**.

O problema é que o seu código **termina na linha 32**. Até essa linha, você calculou a velocidade (`hspd`), mas **não mandou o personagem sair do lugar** e nem aplicou a gravidade. É como se você tivesse acelerado o carro, mas não soltado o freio de mão.

Você precisa colar o restante do código **logo abaixo da linha 32** no evento **Etapa (Step)**.

Aqui está a parte que falta. Copie e cole isso no final do seu código atual:

```
// ... (seu código anterior termina aqui: hspd = _move * walk_spd;)

// =========================================================
// A PARTE QUE FALTA: GRAVIDADE E MOVIMENTO REAL
// =========================================================

// 1. Aplicar Gravidade (Aumenta a velocidade de queda)
vspd = vspd + grav;
```

```
// 2. Pulo (Só se estiver tocando no chão E apertar o botão)
if (place_meeting(x, y + 1, obj_chao) && _key_jump)
{
    vspd = -jump_spd;
}

// 3. Colisão Horizontal e Movimento X
// Verifica se vai bater na parede
if (place_meeting(x + hspd, y, obj_chao))
{
    while (!place_meeting(x + sign(hspd), y, obj_chao))
    {
        x = x + sign(hspd);
    }
    hspd = 0;
}
x = x + hspd; // <--- ESSA LINHA FAZ ELE ANDAR PARA OS LADOS

// 4. Colisão Vertical e Movimento Y
// Verifica se vai bater no chão ou teto
if (place_meeting(x, y + vspd, obj_chao))
{
    while (!place_meeting(x, y + sign(vspd), obj_chao))
    {
        y = y + sign(vspd);
    }
    vspd = 0;
}
y = y + vspd; // <--- ESSA LINHA FAZ ELE CAIR OU PULAR

// =========================================================
// ANIMAÇÃO E ESPELHAMENTO
// =========================================================

if (hspd != 0) // Se estiver se movendo
{
    sprite_index = spr_bit_walk; // Troca para sprite andando
    image_xscale = sign(hspd);   // Vira para o lado (1 ou -1)
}
else // Se estiver parado
{
    sprite_index = spr_bit_stand; // Troca para sprite parado
}
```

⚠️ **Um ajuste importante nos Sprites**

Para o comando `image_xscale = -1` (virar para a esquerda) funcionar sem o personagem "teletransportar" levemente para o lado:

1. Abra o sprite `spr_bit_walk`.
2. Na configuração de **Origin** (Origem), mude de "Top Left" para **Middle Centre** (Centro Meio) ou **Bottom Centre** (Centro Baixo).
3. Faça o mesmo no `spr_bit_stand`.

Isso garante que ele gire em torno do próprio umbigo, e não em torno do canto da imagem.

Tente adicionar esse código e me diga se ele começou a se mexer!

```
#########################################
#########################################
####################
ERROR in action number 1
of  Step Event0 for object obj_bit:...
```

Mostrar raciocínio

Esse erro significa que **o GameMaker não encontrou nenhum Sprite com o nome** `spr_bit_walk`.

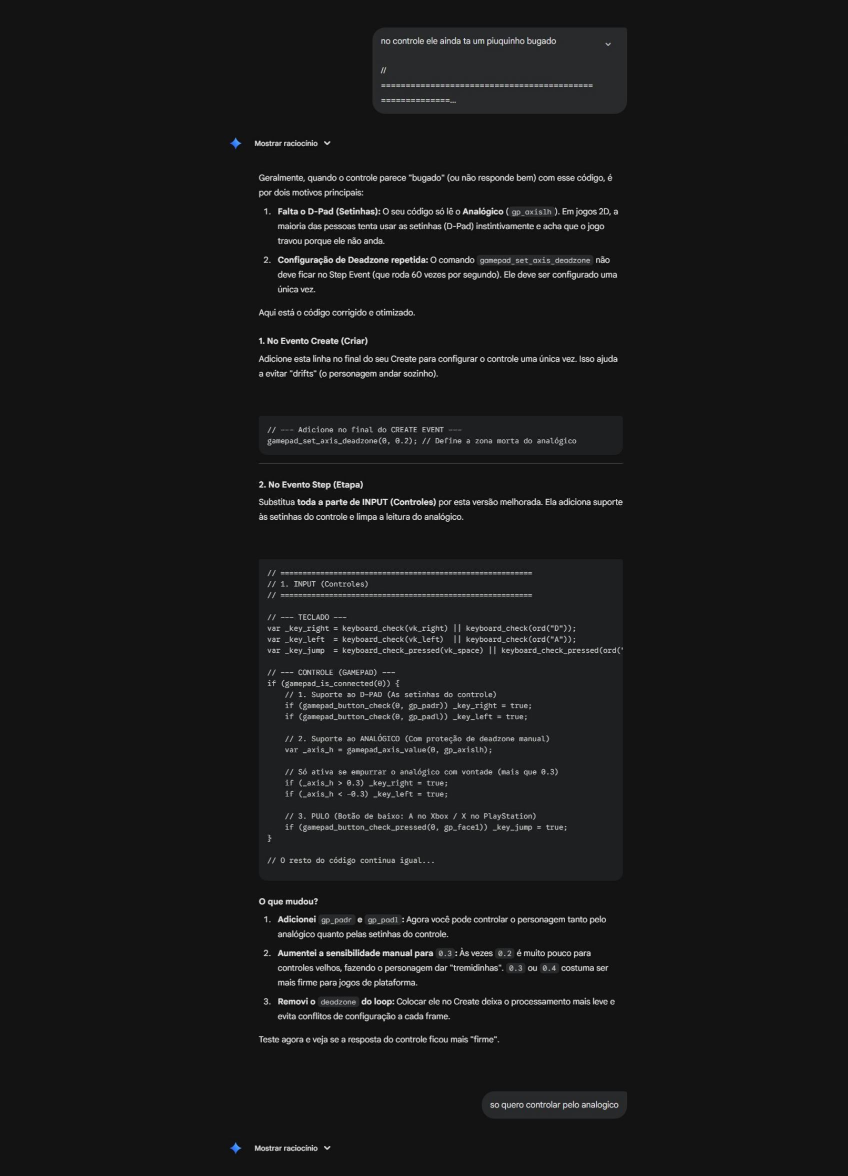
Quando o GameMaker não encontra um "Recurso" (como um som ou imagem) com o nome que você escreveu, ele acha que você está tentando usar uma variável que esqueceu de criar.

Como consertar:

1. Olhe para a direita da sua tela, no **Navegador de Ativos (Asset Browser)**.
2. Abra a pastinha **Sprites**.
3. Confira **exatamente** como você escreveu o nome da animação de andar.
    - Você escreveu `spr_bit_run` em vez de `spr_bit_walk`?
    - Faltou algum underline? (Ex: `sprbit_walk`)?
    - Está escrito `spr_bit_walk ` (com um espaço no final)?
4. **Volte ao código** (linha 71) e escreva o nome **exatamente igual** ao que está na pasta.

Se o nome ficar **vermelho ou laranja** no código, significa que o GameMaker reconheceu o sprite. Se ficar **azul claro ou cinza**, ele ainda não reconheceu (ou o sprite não existe).

Dica: Se você começar a digitar `spr_` e esperar um pouco (ou apertar `Ctrl + Espaço`), o GameMaker abre uma lista com todos os sprites que existem. Selecione o correto na lista para evitar erros de digitação.



no controle ele ainda ta um piuquinho bugado

//

==

===============...

Geralmente, quando o controle parece "bugado" (ou não responde bem) com esse código, é por dois motivos principais:

1. **Falta o D-Pad (Setinhas):** O seu código só lê o **Analógico** (`gp_axislh`). Em jogos 2D, a maioria das pessoas tenta usar as setinhas (D-Pad) instintivamente e acha que o jogo travou porque ele não anda.
2. **Configuração de Deadzone repetida:** O comando `gamepad_set_axis_deadzone` não deve ficar no Step Event (que roda 60 vezes por segundo). Ele deve ser configurado uma única vez.

Aqui está o código corrigido e otimizado.

1. No Evento Create (Criar)

Adicione esta linha no final do seu Create para configurar o controle uma única vez. Isso ajuda a evitar "drifts" (o personagem andar sozinho).

```
// --- Adicione no final do CREATE EVENT ---
gamepad_set_axis_deadzone(0, 0.2); // Define a zona morta do analógico
```

---

2. No Evento Step (Etapa)

Substitua **toda a parte de INPUT (Controles)** por esta versão melhorada. Ela adiciona suporte às setinhas do controle e limpa a leitura do analógico.

```
// =========================================================
// 1. INPUT (Controles)
// =========================================================

// --- TECLADO ---
var _key_right = keyboard_check(vk_right) || keyboard_check(ord("D"));
var _key_left  = keyboard_check(vk_left)  || keyboard_check(ord("A"));
var _key_jump  = keyboard_check_pressed(vk_space) || keyboard_check_pressed(ord('

// --- CONTROLE (GAMEPAD) ---
if (gamepad_is_connected(0)) {
    // 1. Suporte ao D-PAD (As setinhas do controle)
    if (gamepad_button_check(0, gp_padr)) _key_right = true;
    if (gamepad_button_check(0, gp_padl)) _key_left = true;

    // 2. Suporte ao ANALÓGICO (Com proteção de deadzone manual)
    var _axis_h = gamepad_axis_value(0, gp_axislh);

    // Só ativa se empurrar o analógico com vontade (mais que 0.3)
    if (_axis_h > 0.3) _key_right = true;
    if (_axis_h < -0.3) _key_left = true;

    // 3. PULO (Botão de baixo: A no Xbox / X no PlayStation)
    if (gamepad_button_check_pressed(0, gp_face1)) _key_jump = true;
}

// O resto do código continua igual...
```

O que mudou?

1. **Adicionei** `gp_padr` **e** `gp_padl` **:** Agora você pode controlar o personagem tanto pelo analógico quanto pelas setinhas do controle.
2. **Aumentei a sensibilidade manual para** `0.3` **:** Às vezes `0.2` é muito pouco para controles velhos, fazendo o personagem dar "tremidinhas". `0.3` ou `0.4` costuma ser mais firme para jogos de plataforma.
3. **Removi o** `deadzone` **do loop:** Colocar ele no Create deixa o processamento mais leve e evita conflitos de configuração a cada frame.

Teste agora e veja se a resposta do controle ficou mais "firme".

so quero controlar pelo analogico

Entendido. Vamos limpar o código para focar **apenas no analógico** (e teclado) e deixar a resposta mais firme.

Para resolver a sensação de "bugado" (o personagem andar sozinho ou travar), o segredo é o **Deadzone** (Zona Morta) no evento **Create**.

1. No Evento Create (Criar)

Se você ainda não colocou, adicione esta linha no final do evento **Criar**. Isso diz para o controle ignorar movimentos muito leves (quando o analógico está velho e meio solto).

```
// Adicione no final do evento CRIAR
gamepad_set_axis_deadzone(0, 0.2);
```

---

2. No Evento Step (Etapa)

Substitua **toda a parte 1. INPUT** por este código aqui. Ele só lê o analógico e ignora as setinhas do controle.

```
// =========================================================
// 1. INPUT (Controles)
// =========================================================

// Configuração Teclado (Mantemos para testes no PC)
var _key_right = keyboard_check(vk_right) || keyboard_check(ord("D"));
var _key_left  = keyboard_check(vk_left)  || keyboard_check(ord("A"));
var _key_jump  = keyboard_check_pressed(vk_space) || keyboard_check_pressed(ord('

// Configuração Controle (APENAS ANALÓGICO)
if (gamepad_is_connected(0)) {

    // Leitura do Eixo Horizontal (Analógico Esquerdo)
    // O valor vai de -1 (esquerda total) até 1 (direita total)
    var _axis_val = gamepad_axis_value(0, gp_axislh);

    // Se empurrar para direita (maior que 0.2 para evitar drift)
    if (_axis_val > 0.2) {
        _key_right = true;
    }

    // Se empurrar para esquerda (menor que -0.2)
    if (_axis_val < -0.2) {
        _key_left = true;
    }

    // Botão de pulo (A no Xbox / X no Playstation)
    if (gamepad_button_check_pressed(0, gp_face1)) {
        _key_jump = true;
    }
}

// O restante do código (Movimento, Física, Colisão) continua igual abaixo...
```

Por que isso resolve: Ao verificar se o valor é maior que `0.2` ou menor que `-0.2`, garantimos que o personagem só ande se você realmente empurrar o dedo. Se o controle estiver parado no centro (que às vezes fica em 0.05 ou -0.05 por desgaste), o código ignora e o personagem fica perfeitamente parado.

//
==
==============
// 1. INPUT (Controles)
//...

Mostrar raciocínio

Entendi. Vamos resolver esses dois problemas:

1. **O "A" do teclado pulando/interferindo:** Se você quer focar no controle, vamos limpar os comandos de teclado para não haver conflito. O "A" do teclado deveria ser esquerda, mas se está pulando, pode haver algum conflito estranho ou confusão com o botão "A" do controle (que é o de pulo). Vamos remover o teclado para garantir.
2. **Bugar ao mudar de direção:** Isso acontece porque quando você vira o personagem (`image_xscale`), a caixa de colisão muda de lugar ou de tamanho se o sprite não estiver perfeitamente centralizado. **A solução profissional** é travar a máscara de colisão para que ela nunca mude, independente da animação.

Aqui está o que você precisa fazer:

Passo 1: Travar a Máscara de Colisão (Para parar de bugar na parede)

Para o personagem não ficar preso na parede quando vira:

1. Abra o seu objeto `obj_bit`.

2. Na janela de propriedades dele, procure a opção **Collision Mask** (Máscara de Colisão).
3. Provavelmente está escrito "Same as sprite" (Igual ao sprite). **Mude isso.**
4. Selecione o sprite `spr_bit_stand` (ou o `spr_colisao` se você tiver um para o personagem).

Isso garante que, mesmo que a animação de andar seja mais larga ou torta, a "caixa física" do personagem nunca muda de tamanho.

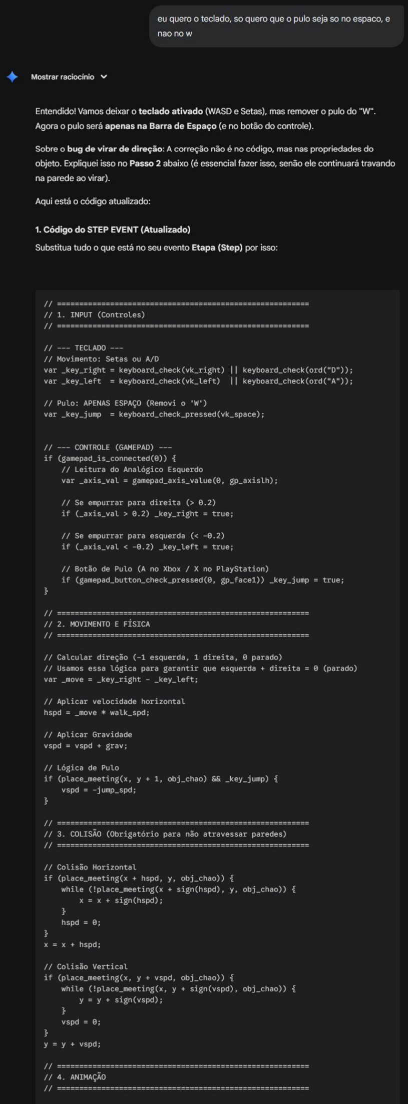
Passo 2: O Código Limpo (Sem teclado atrapalhando)

Substitua todo o seu código do **Step Event** por este. Removi o teclado e ajustei a sensibilidade.

```
// ==========================================================
// 1. INPUT (Apenas Controle Analógico)
// ==========================================================

// Inicializa as variáveis como falso
var _key_right = false;
var _key_left  = false;
var _key_jump  = false;

// Verifica se o controle está conectado (Slot 0)
if (gamepad_is_connected(0)) {
    // Leitura do Analógico Esquerdo
    var _axis_val = gamepad_axis_value(0, gp_axislh);

    // Movimento (com deadzone manual de 0.2)
    if (_axis_val > 0.2) _key_right = true;
    if (_axis_val < -0.2) _key_left = true;

    // Pulo (Botão A do Xbox / X do PlayStation)
    if (gamepad_button_check_pressed(0, gp_face1)) _key_jump = true;
}

// ==========================================================
// 2. MOVIMENTO E FÍSICA
// ==========================================================

// Calcular direção
var _move = 0;
if (_key_right) _move = 1;
if (_key_left) _move = -1;

// Aplicar velocidade horizontal
hspd = _move * walk_spd;

// Aplicar Gravidade
vspd = vspd + grav;

// Lógica de Pulo
if (place_meeting(x, y + 1, obj_chao) && _key_jump) {
    vspd = -jump_spd;
}

// ==========================================================
// 3. COLISÃO
// ==========================================================

// Horizontal
if (place_meeting(x + hspd, y, obj_chao)) {
    while (!place_meeting(x + sign(hspd), y, obj_chao)) {
        x = x + sign(hspd);
    }
    hspd = 0;
}
x = x + hspd;

// Vertical
if (place_meeting(x, y + vspd, obj_chao)) {
    while (!place_meeting(x, y + sign(vspd), obj_chao)) {
        y = y + sign(vspd);
    }
    vspd = 0;
}
y = y + vspd;

// ==========================================================
// 4. ANIMAÇÃO
// ==========================================================

if (hspd != 0) {
    sprite_index = spr_bit_walk;
    image_xscale = sign(hspd);
}
else {
    sprite_index = spr_bit_stand;
}
```

Resumo das correções:

1. **Removi o Teclado:** As linhas `keyboard_check` sumiram. Agora o teclado não interfere.
2. **Correção do Bug de Virar:** Além do código, **faça o Passo 1** (fixar a Collision Mask no objeto). É isso que impede ele de entrar na parede quando vira.
3. **Variável `_move` mais segura:** Mudei a lógica de `_key_right - _key_left` para `if/else` simples, o que evita erros matemáticos se o controle falhar por um milissegundo.



eu quero o teclado, so quero que o pulo seja so no espaco, e nao no w

Entendido! Vamos deixar o **teclado ativado** (WASD e Setas), mas remover o pulo do "W". Agora o pulo será **apenas na Barra de Espaço** (e no botão do controle).

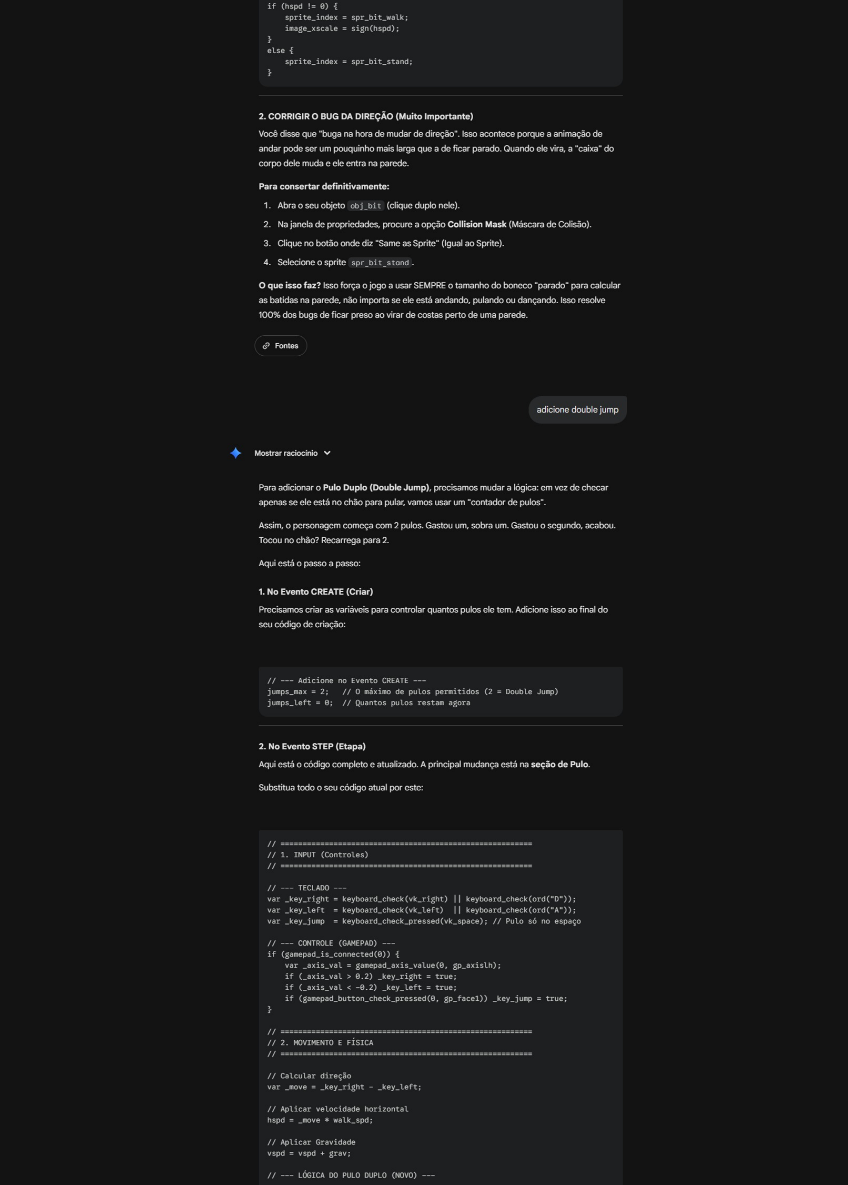
Sobre o **bug de virar de direção:** A correção não é no código, mas nas propriedades do objeto. Expliquei isso no **Passo 2** abaixo (é essencial fazer isso, senão ele continuará travando na parede ao virar).

Aqui está o código atualizado:

1. Código do STEP EVENT (Atualizado)

Substitua tudo o que está no seu evento **Etapa (Step)** por isso:

```
// ========================================================
// 1. INPUT (Controles)
// ========================================================

// --- TECLADO ---
// Movimento: Setas ou A/D
var _key_right = keyboard_check(vk_right) || keyboard_check(ord("D"));
var _key_left  = keyboard_check(vk_left)  || keyboard_check(ord("A"));

// Pulo: APENAS ESPAÇO (Removi o 'W')
var _key_jump  = keyboard_check_pressed(vk_space);


// --- CONTROLE (GAMEPAD) ---
if (gamepad_is_connected(0)) {
    // Leitura do Analógico Esquerdo
    var _axis_val = gamepad_axis_value(0, gp_axislh);

    // Se empurrar para direita (> 0.2)
    if (_axis_val > 0.2) _key_right = true;

    // Se empurrar para esquerda (< -0.2)
    if (_axis_val < -0.2) _key_left = true;

    // Botão de Pulo (A no Xbox / X no PlayStation)
    if (gamepad_button_check_pressed(0, gp_face1)) _key_jump = true;
}

// ========================================================
// 2. MOVIMENTO E FÍSICA
// ========================================================

// Calcular direção (-1 esquerda, 1 direita, 0 parado)
// Usamos essa lógica para garantir que esquerda + direita = 0 (parado)
var _move = _key_right - _key_left;

// Aplicar velocidade horizontal
hspd = _move * walk_spd;

// Aplicar Gravidade
vspd = vspd + grav;

// Lógica de Pulo
if (place_meeting(x, y + 1, obj_chao) && _key_jump) {
    vspd = -jump_spd;
}

// ========================================================
// 3. COLISÃO (Obrigatório para não atravessar paredes)
// ========================================================

// Colisão Horizontal
if (place_meeting(x + hspd, y, obj_chao)) {
    while (!place_meeting(x + sign(hspd), y, obj_chao)) {
        x = x + sign(hspd);
    }
    hspd = 0;
}
x = x + hspd;

// Colisão Vertical
if (place_meeting(x, y + vspd, obj_chao)) {
    while (!place_meeting(x, y + sign(vspd), obj_chao)) {
        y = y + sign(vspd);
    }
    vspd = 0;
}
y = y + vspd;

// ========================================================
// 4. ANIMAÇÃO
// ========================================================
```



```
if (hspd != 0) {
    sprite_index = spr_bit_walk;
    image_xscale = sign(hspd);
}
else {
    sprite_index = spr_bit_stand;
}
```

---

2. CORRIGIR O BUG DA DIREÇÃO (Muito Importante)

Você disse que "buga na hora de mudar de direção". Isso acontece porque a animação de andar pode ser um pouquinho mais larga que a de ficar parado. Quando ele vira, a "caixa" do corpo dele muda e ele entra na parede.

Para consertar definitivamente:

1. Abra o seu objeto `obj_bit` (clique duplo nele).
2. Na janela de propriedades, procure a opção **Collision Mask** (Máscara de Colisão).
3. Clique no botão onde diz "Same as Sprite" (Igual ao Sprite).
4. Selecione o sprite `spr_bit_stand`.

O que isso faz? Isso força o jogo a usar SEMPRE o tamanho do boneco "parado" para calcular as batidas na parede, não importa se ele está andando, pulando ou dançando. Isso resolve 100% dos bugs de ficar preso ao virar de costas perto de uma parede.

adicione double jump

Para adicionar o **Pulo Duplo (Double Jump)**, precisamos mudar a lógica: em vez de checar apenas se ele está no chão para pular, vamos usar um "contador de pulos".

Assim, o personagem começa com 2 pulos. Gastou um, sobra um. Gastou o segundo, acabou. Tocou no chão? Recarrega para 2.

Aqui está o passo a passo:

1. No Evento CREATE (Criar)

Precisamos criar as variáveis para controlar quantos pulos ele tem. Adicione isso ao final do seu código de criação:

```
// --- Adicione no Evento CREATE ---
jumps_max = 2;   // O máximo de pulos permitidos (2 = Double Jump)
jumps_left = 0;  // Quantos pulos restam agora
```

---

2. No Evento STEP (Etapa)

Aqui está o código completo e atualizado. A principal mudança está na **seção de Pulo**.

Substitua todo o seu código atual por este:

```
// =========================================================
// 1. INPUT (Controles)
// =========================================================

// --- TECLADO ---
var _key_right = keyboard_check(vk_right) || keyboard_check(ord("D"));
var _key_left  = keyboard_check(vk_left)  || keyboard_check(ord("A"));
var _key_jump  = keyboard_check_pressed(vk_space); // Pulo só no espaço

// --- CONTROLE (GAMEPAD) ---
if (gamepad_is_connected(0)) {
    var _axis_val = gamepad_axis_value(0, gp_axislh);
    if (_axis_val > 0.2) _key_right = true;
    if (_axis_val < -0.2) _key_left = true;
    if (gamepad_button_check_pressed(0, gp_face1)) _key_jump = true;
}

// =========================================================
// 2. MOVIMENTO E FÍSICA
// =========================================================

// Calcular direção
var _move = _key_right - _key_left;

// Aplicar velocidade horizontal
hspd = _move * walk_spd;

// Aplicar Gravidade
vspd = vspd + grav;

// --- LÓGICA DO PULO DUPLO (NOVO) ---
```

```
// Se estiver no chão, recarrega os pulos
if (place_meeting(x, y + 1, obj_chao)) {
    jumps_left = jumps_max;
}

// Se apertar pular E ainda tiver pulos sobrando
if (_key_jump && jumps_left > 0) {
    vspd = -jump_spd; // Aplica o pulo
    jumps_left -= 1;  // Gasta um pulo do estoque
}

// =========================================================
// 3. COLISÃO
// =========================================================

// Horizontal
if (place_meeting(x + hspd, y, obj_chao)) {
    while (!place_meeting(x + sign(hspd), y, obj_chao)) {
        x = x + sign(hspd);
    }
    hspd = 0;
}
x = x + hspd;

// Vertical
if (place_meeting(x, y + vspd, obj_chao)) {
    while (!place_meeting(x, y + sign(vspd), obj_chao)) {
        y = y + sign(vspd);
    }
    vspd = 0;
}
y = y + vspd;

// =========================================================
// 4. ANIMAÇÃO
// =========================================================

if (hspd != 0) {
    sprite_index = spr_bit_walk;
    image_xscale = sign(hspd);
}
else {
    sprite_index = spr_bit_stand;
}
```

O que mudou?

1. **Recarga:** Quando ele toca o chão (`place_meeting`), a variável `jumps_left` enche de volta para 2.
2. **Consumo:** Quando você aperta pular, o código agora pergunta: "Tenho pulos maior que zero?". Se sim, ele pula e desconta 1.

Agora ele deve conseguir pular do chão e dar mais um pulo no ar!

quero o jogo em tela cheia, como fazer isso?

Mostrar raciocínio

Existem duas formas principais de fazer isso: configurando o jogo para **já abrir em tela cheia** ou criando um **botão para alternar** (como o F11 nos navegadores).

Opção 1: Já começar em Tela Cheia (Configuração)

Essa é a forma mais fácil, sem precisar de código.

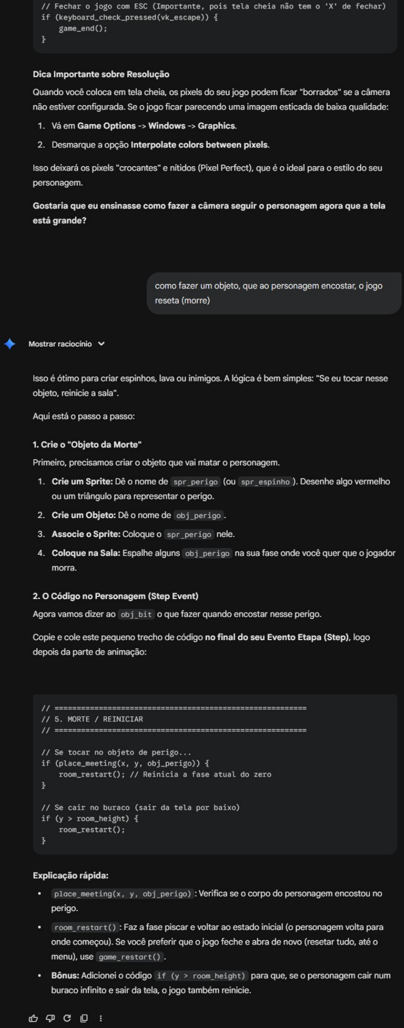
1. No menu superior do GameMaker, procure um ícone de "engrenagem" ou vá em **Game Options** (Opções do Jogo).
2. Na janela que abrir, selecione **Windows** na lista à esquerda.
3. Clique na aba **Graphics** (Gráficos).
4. Marque a caixinha **Start Fullscreen** (Iniciar em Tela Cheia).
5. Dê OK e teste o jogo.

Opção 2: Alternar via Código (Botão F11)

Se você quiser que o jogador possa entrar e sair da tela cheia apertando uma tecla (recomendado), adicione isso no **Step Event** do seu `obj_bit` (ou de um objeto controlador do jogo):

Cole isso no final do seu Step Event:

```
// --- TELA CHEIA E SAIR ---

// Alternar Tela Cheia com F11
if (keyboard_check_pressed(vk_f11)) {
    window_set_fullscreen(!window_get_fullscreen());
}
```



```
// Fechar o jogo com ESC (Importante, pois tela cheia não tem o 'X' de fechar)
if (keyboard_check_pressed(vk_escape)) {
    game_end();
}
```

Dica Importante sobre Resolução

Quando você coloca em tela cheia, os pixels do seu jogo podem ficar "borrados" se a câmera não estiver configurada. Se o jogo ficar parecendo uma imagem esticada de baixa qualidade:

1. Vá em **Game Options** -> **Windows** -> **Graphics**.
2. Desmarque a opção **Interpolate colors between pixels**.

Isso deixará os pixels "crocantes" e nítidos (Pixel Perfect), que é o ideal para o estilo do seu personagem.

Gostaria que eu ensinasse como fazer a câmera seguir o personagem agora que a tela está grande?

como fazer um objeto, que ao personagem encostar, o jogo reseta (morre)

Isso é ótimo para criar espinhos, lava ou inimigos. A lógica é bem simples: "Se eu tocar nesse objeto, reinicie a sala".

Aqui está o passo a passo:

1. Crie o "Objeto da Morte"

Primeiro, precisamos criar o objeto que vai matar o personagem.

1. **Crie um Sprite:** Dê o nome de `spr_perigo` (ou `spr_espinho`). Desenhe algo vermelho ou um triângulo para representar o perigo.
2. **Crie um Objeto:** Dê o nome de `obj_perigo`.
3. **Associe o Sprite:** Coloque o `spr_perigo` nele.
4. **Coloque na Sala:** Espalhe alguns `obj_perigo` na sua fase onde você quer que o jogador morra.

2. O Código no Personagem (Step Event)

Agora vamos dizer ao `obj_bit` o que fazer quando encostar nesse perigo.

Copie e cole este pequeno trecho de código **no final do seu Evento Etapa (Step)**, logo depois da parte de animação:

```
// =====================================================
// 5. MORTE / REINICIAR
// =====================================================

// Se tocar no objeto de perigo...
if (place_meeting(x, y, obj_perigo)) {
    room_restart(); // Reinicia a fase atual do zero
}

// Se cair no buraco (sair da tela por baixo)
if (y > room_height) {
    room_restart();
}
```

Explicação rápida:

- `place_meeting(x, y, obj_perigo)`: Verifica se o corpo do personagem encostou no perigo.
- `room_restart()`: Faz a fase piscar e voltar ao estado inicial (o personagem volta para onde começou). Se você preferir que o jogo feche e abra de novo (resetar tudo, até o menu), use `game_restart()`.
- **Bônus:** Adicionei o código `if (y > room_height)` para que, se o personagem cair num buraco infinito e sair da tela, o jogo também reinicie.